



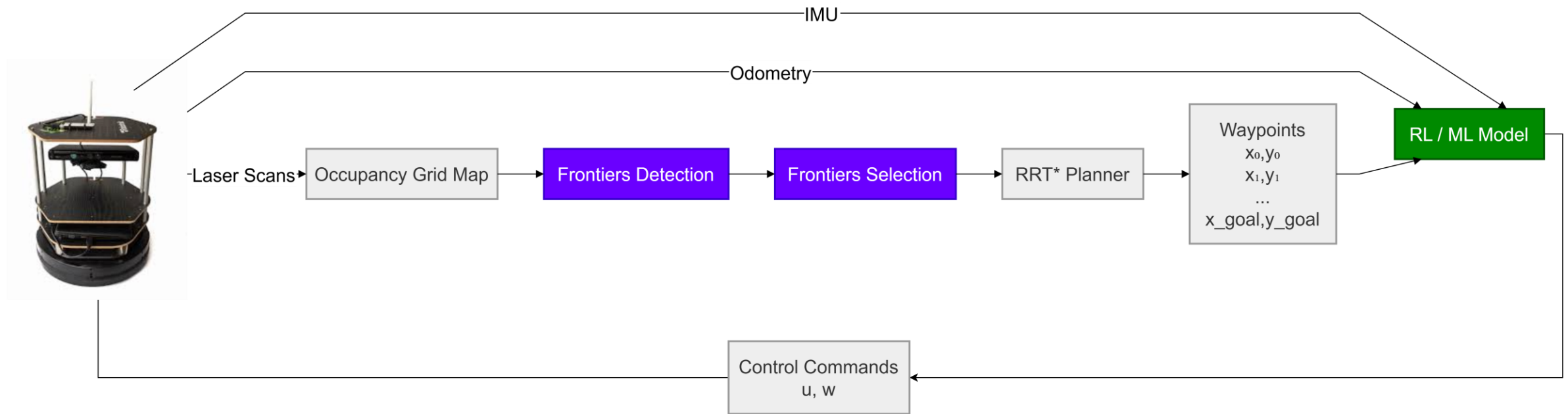
HANDS ON PLANNING

Exploration global-local planner: with RL

Huu Truong Giang Nguyen
Elchin Aslanli

- 1. Conceptual design**
- 2. Frontier based exploration algorithm**
- 3. DRL Local Planner**
- 4. Results**
- 5. Conclusion**

Conceptual Design



Frontier Based Exploration Algorithm

1. System check

Map, odometry, and startup spin must be ready.

2. Failure handling

Keep the goal if the robot is progressing. Reset it if reached, timed out, or stuck. Blacklist unreachable or low-progress frontier clusters, except when only one cluster remains. If all options are blocked by blacklist, retry once without blacklist filtering.

3. Frontier extraction

Detect free cells next to unknown space, then keep only reachable frontier cells.

5. Cluster evaluation

Group frontiers into clusters and compute each cluster centroid.

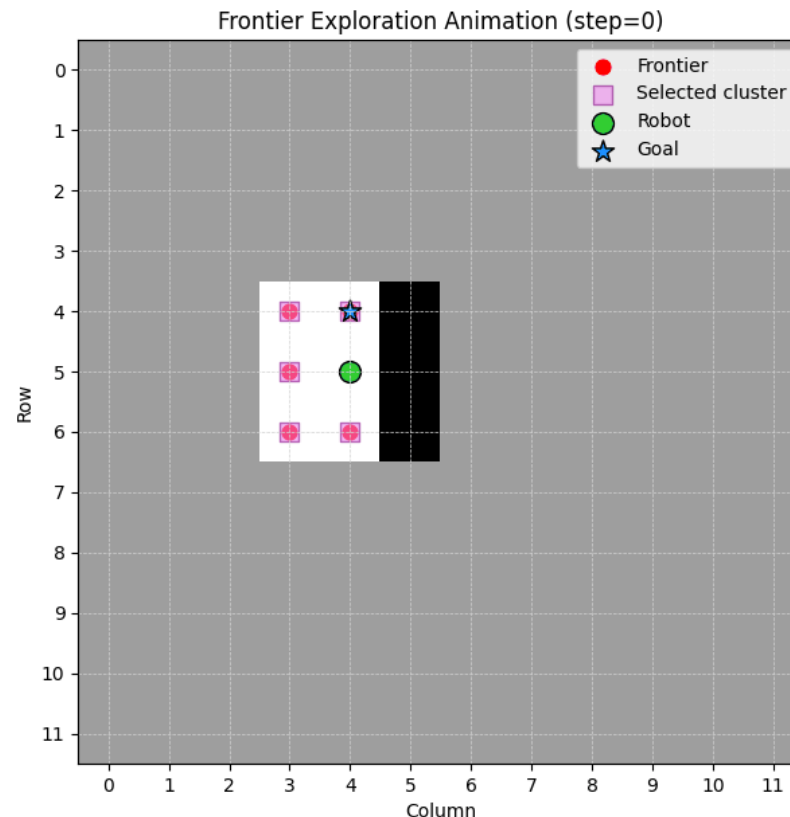
Score clusters using:

Information gain – Distance + Size of cluster.

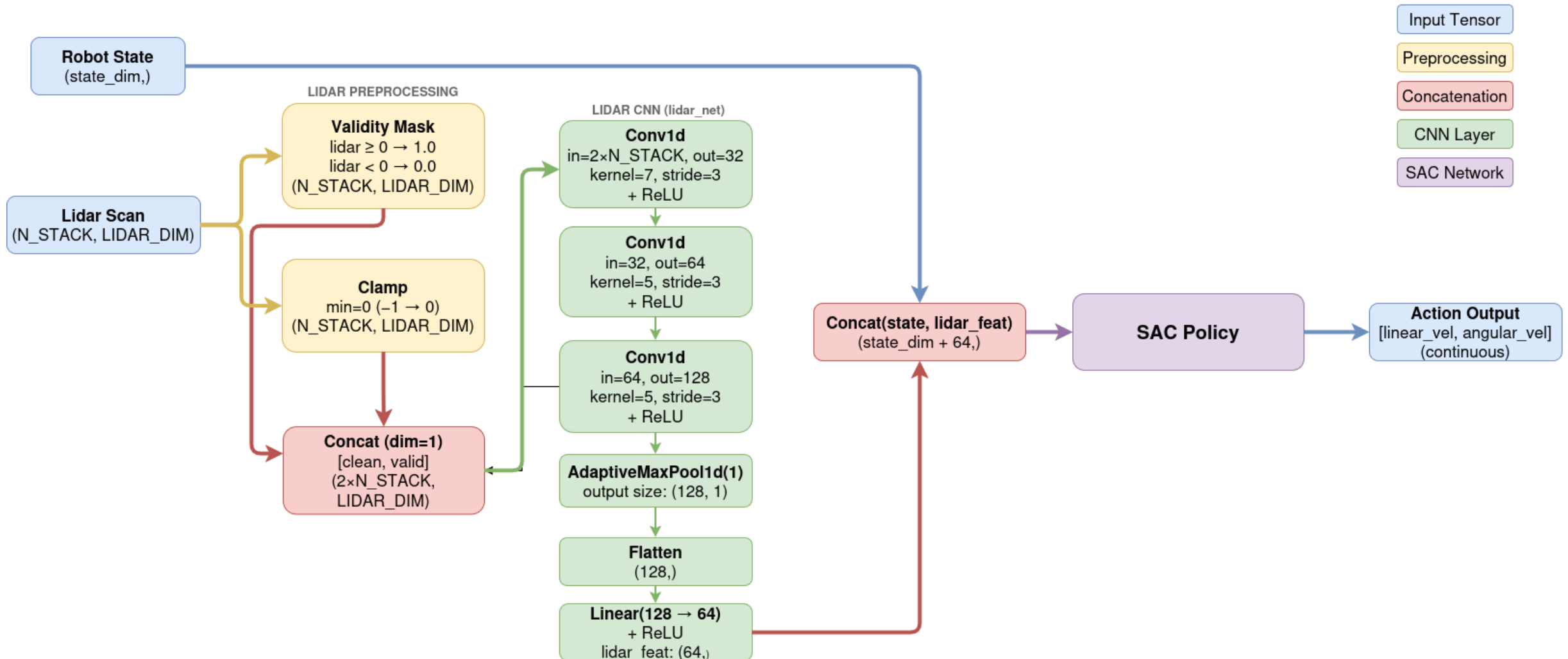
6. Safe goal generation

Select a candidate frontier from the best cluster.

Shift it inward toward free space and validate clearance, map borders, unknown distance, and inflated-map safety.



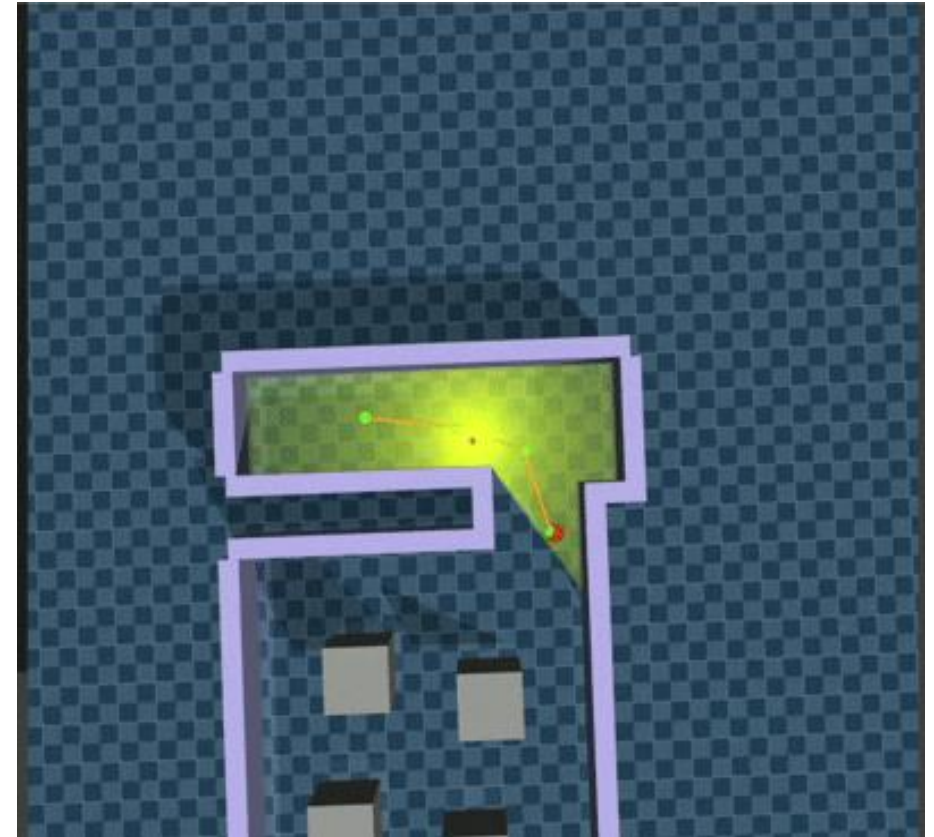
TurtleBot2 SAC Network Architecture



Reward design:

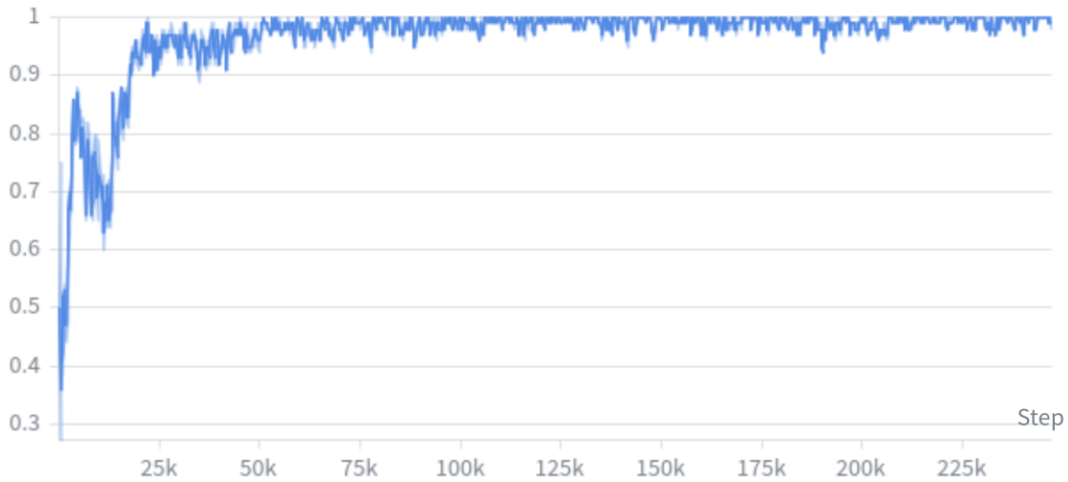
1. Go-Straight Reward
2. Waypoint Distance Progress Reward
3. Heading Angle Progress Reward
4. Velocity Smoothness Penalty
5. Collision Penalty
6. Obstacle Proximity Penalty
7. Path Following Reward
8. Turn-or-Drive Behavior Reward
9. Waypoint Reaching Bonus
10. Final Goal Reward

$$R_{\text{total}} = 5r_1 + 3r_2 + 0.5r_3 + 0.3r_4 + r_5 + 0.2r_6 + 0.02r_7 + 1.5r_8 + r_9 + r_{10}$$

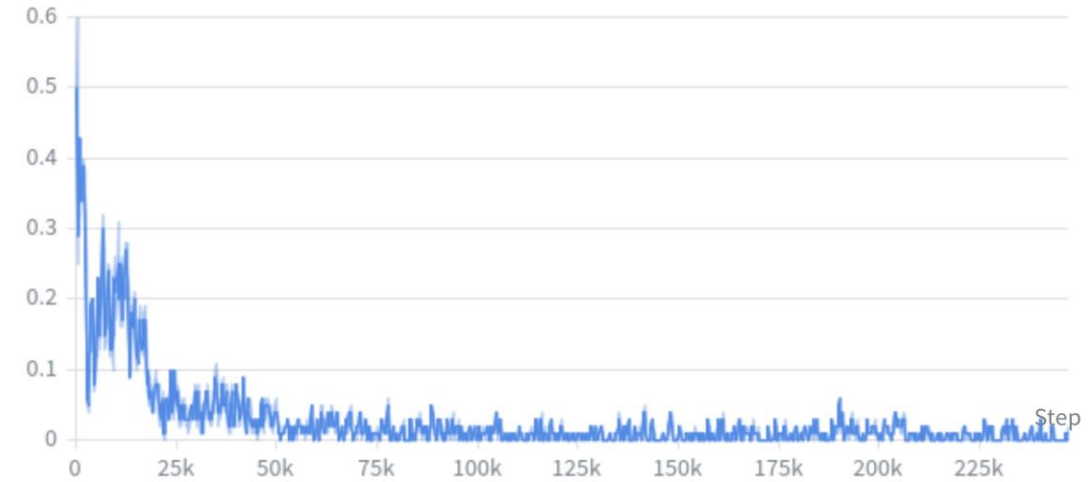


DRL Chart

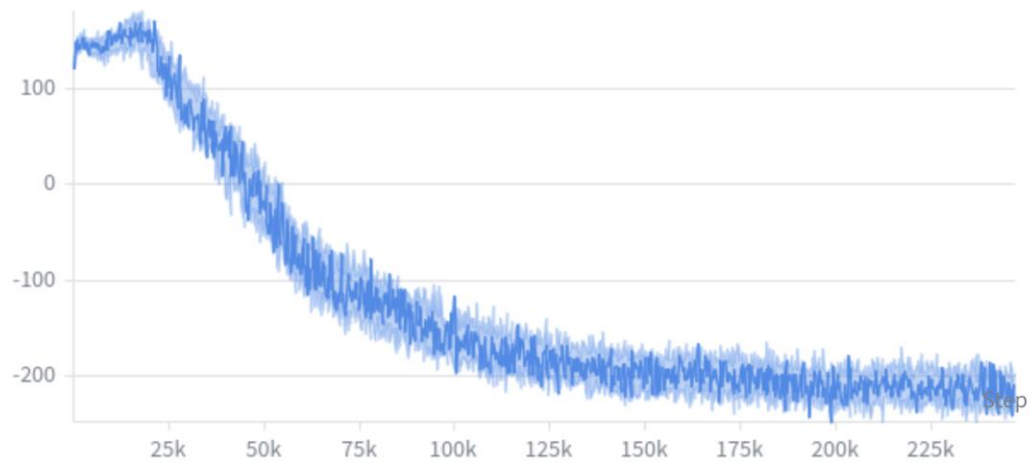
episode/goal_reached_rate



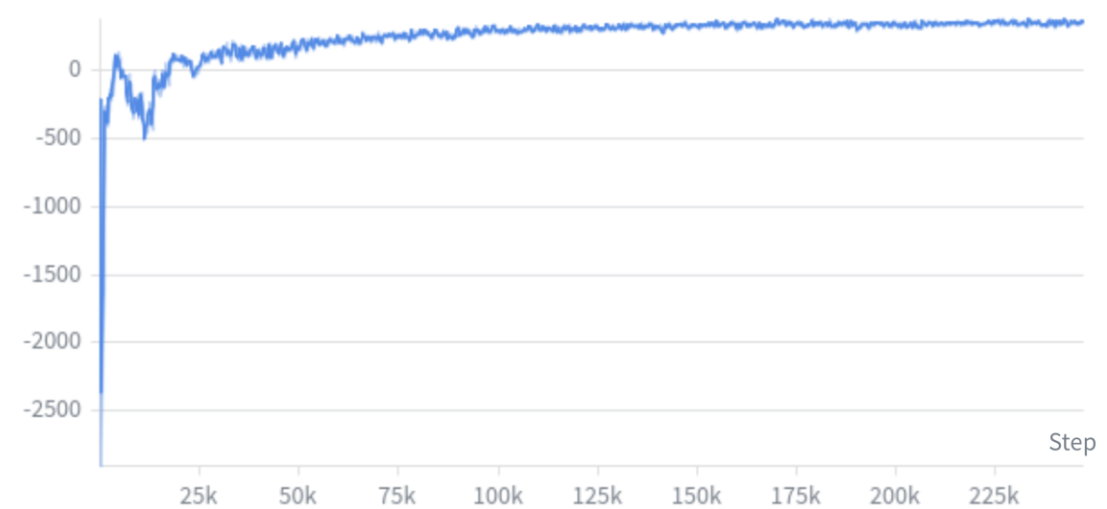
episode/collision_rate



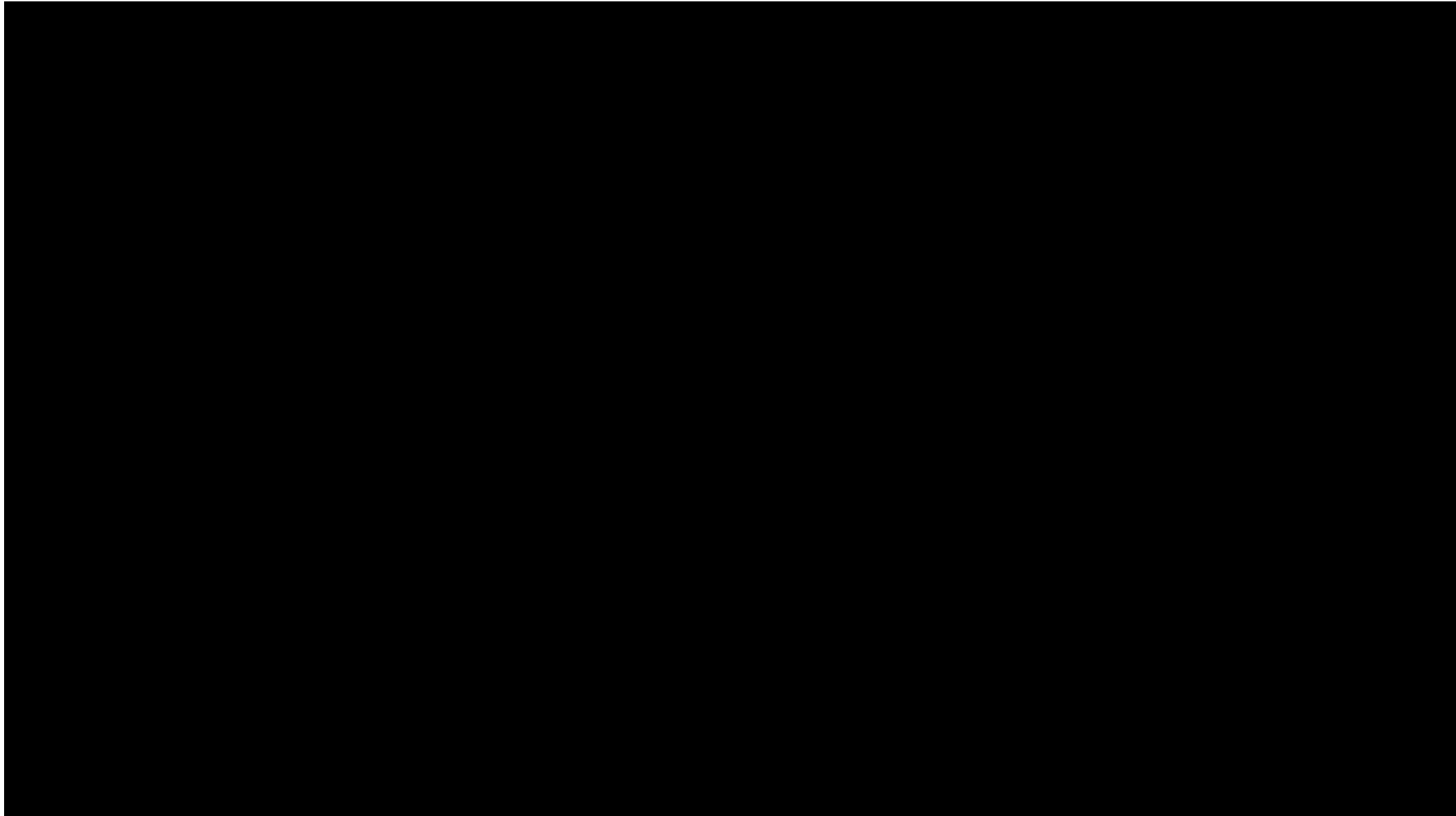
train/actor_loss



rollout/ep_rew_mean



Results



Conclusion:

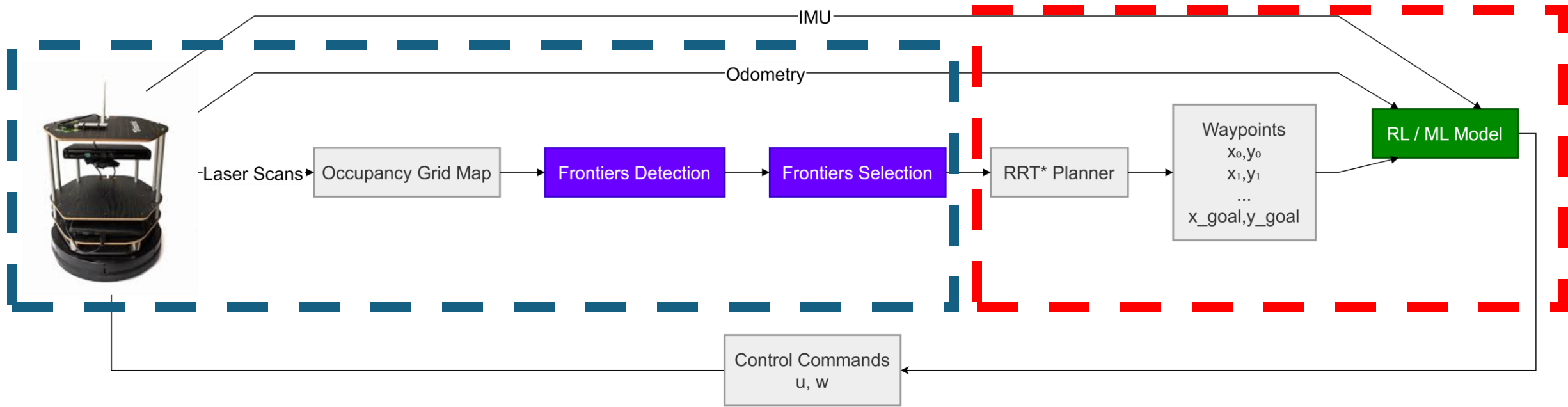
The proposed global-local planner successfully combines **frontier-based exploration** with a **DRL local planner** and works on a real robot. This shows that the system can select useful exploration goals and generate safe motion commands in real-world conditions.

Future work:

We would like to do a **CI-CD** system, that robot continuously learn from each movement in real life, not just simulation.

TASK DIVISION

ELCHIN
GIANG



Frontier Based Exploration
Tests with Pose Controller
Integration of RL model to real robot

RL Model training
RRT* Planner
Model tests in Mujoco environment

Straight Motion Reward

$$r_{\text{straight}} = \min(v_{\text{lin}}, 0)$$

Progress Toward Waypoint

$$r_{\text{dist}} = d_{t-1} - d_t, \quad d_t = \|p_t - w_{\text{next}}\|_2$$

Heading Alignment to Waypoint

$$r_{\text{angle}} = |\psi_{t-1}| - |\psi_t|, \quad \psi_t = \text{angle to next waypoint}$$

Smoothness Penalty

$$r_{\text{smooth}} = -(\alpha |\Delta v_{\text{lin}}| + \beta |\Delta \omega|) - \mathbb{I}[\text{direction reversal}]$$

$$\alpha = \beta = 1$$

Collision Penalty

$$r_{\text{collision}} = \begin{cases} -500, & \text{if collision occurs} \\ 0, & \text{otherwise} \end{cases}$$

Obstacle Proximity Penalty

$$r_{\text{obstacle}} = \begin{cases} -\frac{1}{\min(\text{lidar})}, & \min(\text{lidar}) < 1.0 \text{ m} \\ 0, & \text{otherwise} \end{cases}$$

Path Following Reward

$$r_{\text{path}} = -\alpha d_{\perp} - \beta |\phi|, \quad \alpha = 0.85, \beta = 0.15$$

Sharp Turn Handling

$$r_{\text{turn}} = \begin{cases} 0.4 |\omega| - 0.8 |v_{\text{lin}}|, & |\psi| > \frac{\pi}{3} \\ 0, & \text{otherwise} \end{cases}$$

Waypoint Bonus

$$r_{\text{wp}} = 50 \cdot \frac{k+1}{N}$$

Terminal Rewards

$$r_{\text{goal}} = +500$$

$$r_{\text{timeout}} = -500$$

Definitions

- $d_{\perp}$: perpendicular distance to the reference path segment
- ϕ : heading error relative to the path direction
- v_{lin} : linear velocity, ω : angular velocity
- k : current waypoint index, N : total number of waypoints

Integrated

